

(◦ ʌ ʌ ◦) NixOS-secrets

Who am I?

- @prescientmoon
- <https://moonythm.dev>
-
-

Who am I?

- @prescientmoon
- <https://moonythm.dev>
- Math :0
- GSOC.

What is this?

What is this?

- Unified interface for managing secrets.
- Pluggable backend system.
-
-
-

What is this?

- Unified interface for managing secrets.
- Pluggable backend system.
- Formerly `nixos-vars` (see: `clan`).
-
-

What is this?

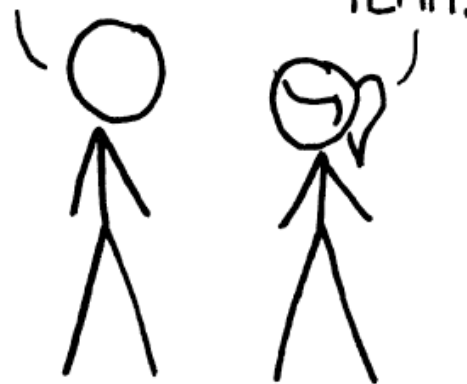
- Unified interface for managing secrets.
- Pluggable backend system.
- Formerly `nixos-vars` (see: `clan`).
- CLI,  NixOS module & beyond :0
- Very experimental (read: might break).

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

The structure of a secret

- A secret holds a non-empty set of files.
- Backends!

The structure of a secret

- A secret holds a non-empty set of files.
- Backends!

```
{  
    secrets.store.cat = {  
        backend = "plain";  
        files.greeting = { };  
    };  
}
```

Generators

Secrets can be generated by a script, no way!

```
{
  secrets.store.cat = {
    # ...
    generate = pkgs: pkgs.writeScript "gen-user" ''
      #!/bin/sh
      echo "Meow~" > "$out/greeting"
    '';
  };
}
```

Dependencies :0

Secrets can depend on each other!

```
{
  secrets.store.cow = {
    dependencies = [ "cat" ];
    files.cow-greeting = { };
    generate = pkgs: pkgs.writeScript "gen-derived" ''
      #!/bin/sh
      export PATH="${lib.makeBinPath [ pkgs.coreutils pkgs.cowsay ]}"
      cat $in/cat/greeting | cowsay > $out/cow-greeting
    '';
  };
}
```

Prompts?!

User input? It's more likely than you think!

Prompts?!

User input? It's more likely than you think!

```
{
  secrets.store.user = {
    prompts.name = {
      backend = "simple";
      label = "Your name";
      description = "The person to address the greeting to";
      type = "line";
    };
    # ...
  };
}
```

Prompts?! (2)

User input? It's more likely than you think!

```
{
  secrets.store.user = {
    # ...
    files.greeting = { };
    generate = pkgs: pkgs.writeScript "gen-user" ''
      #!/bin/sh
      export PATH="${lib.makeBinPath [ pkgs.coreutils ]}"
      echo "Hi! My name is $(cat "$prompts/name")." > "$out/greeting"
    '';
  };
}
```



Act 2: Backends

I guess we're doing acts now...

Prompt backends

- Ask!
- Given secret-name, prompt-name, prompt-type, prompt-label, and prompt-description; writes to \$out.

Prompt backends

- Ask!
- Given secret-name, prompt-name, prompt-type, prompt-label, and prompt-description; writes to \$out.

```
{
  secrets.backends.prompt.simple.ask = pkgs:
    pkgs.writeScript "simple-prompt" ''
      #!/bin/sh
      export PATH="${lib.makeBinPath [ pkgs.coreutils ]}"
      if [[ "$3" == "line" ]]; then
        read -rp "$4: " text
        echo -n "$text" > "$out"
      # ...
    '';
}
```

Store backends

- Actual secret storage.
- Can also handle deployment (when needed).

Get

- Takes a secret and file name, writes the content to \$out

Get

- Takes a secret and file name, writes the content to \$out

Set

- Takes a secret and file name, writes the content of \$in to the secret.

Get

- Takes a secret and file name, writes the content to \$out

Set

- Takes a secret and file name, writes the content of \$in to the secret.

List

- Produces a list of all the secret, file pairs stored by the backend.
- Some might be stale – useful for garbage collection!

Delete

- Deletes the given secret, file pair.

Delete

- Deletes the given secret, file pair.

Fixup

- Invoked for every secret (on each run).

Deployments

Deployments

- `deploy.remote` (whatever you want, really).
- `deploy.local` (the current machine, live CDs, etc).

Deployments

- `deploy.remote` (whatever you want, really).
- `deploy.local` (the current machine, live CDs, etc).

Extensibility

- Backends (of any kind) can just... add options!

Act 3: the CLI

3 as in :3

The CLI's input:

-  Nix files / flakes.
-
-
-
-

The CLI's input:

-  Nix files / flakes.
- can take unevaluated  NixOS configurations.
- can take evaluated  NixOS configurations (ala flakes).
-
-

The CLI's input:

-  Nix files / flakes.
- can take unevaluated  NixOS configurations.
- can take evaluated  NixOS configurations (ala flakes).
- can take  Nix data matching a specific schema.
- can take JSON.

Generating the secrets

- `nixos-secrets generate`
- `nixos-secrets generate --set secret=path/to/stuff`
- `nixos-secrets generate --generate secret`

Generating the secrets

- `nixos-secrets generate`
- `nixos-secrets generate --set secret=path/to/stuff`
- `nixos-secrets generate --generate secret`

Other commands

- `collect-garbage`
-

Generating the secrets

- `nixos-secrets generate`
- `nixos-secrets generate --set secret=path/to/stuff`
- `nixos-secrets generate --generate secret`

Other commands

- `collect-garbage`
- `deploy`



Act 4: Behind the scenes

How high will the counter go?



Stuff can go wrong D:





Stuff can go wrong D:



1. Declare secrets A and B. B depends on A.
2. Generate them successfully!
- 3.
- 4.
- 5.
- 6.



Stuff can go wrong D:



1. Declare secrets A and B. B depends on A.
2. Generate them successfully!
3. Tweak A's script, run the CLI with `--generate a`.
4. Oh no! B's script has a bug that makes it crash :(
- 5.
- 6.



Stuff can go wrong D:



1. Declare secrets A and B. B depends on A.
2. Generate them successfully!
3. Tweak A's script, run the CLI with `--generate a`.
4. Oh no! B's script has a bug that makes it crash :(
5. Fix the bug & run the CLI without `--generate a`.
6. No reason for the CLI to update B; inconsistent state :(



Metadata

- Each secret has a metadata file attached to it.
- Looks like any other file to the secret.
-

Metadata

- Each secret has a metadata file attached to it.
- Looks like any other file to the secret.
- No more bad states!^{1,2,3,4,5}



Act 5: The state of secrets

Is it just me, or are the acts getting shorter?

Example backends

- No production-ready backend (write your own!).
-
-
-
-
-

Example backends

- No production-ready backend (write your own!).
- A simple prompt backend.
- A testing prompt backend.
-
-
-

Example backends

- No production-ready backend (write your own!).
- A simple prompt backend.
- A testing prompt backend.
- A failing store backend.
- A plain-text store backend.
- An age-based store backend.

Missing stuff

- Production backend.
- User/group/permissions.
- Home manager integration.
- The --edit flag.

Act 6: Conclusion?

The last one :0

Thank you

- Ask questions!
- Read through the docs & point out anything that's confusing.
- Try switching (a tiny part) of your config to nixos-secrets.
- Write a custom backend! (for example one that deploys via the Nix store)

- Link to cheatsheet here (with QR code).

